DD2424 Deep Learning: Assignment 3

Linéa Lejerkrans

April 29, 2026

1 Introduction

The goal of this assignment was to implement and train a Convolutional Neural Network (CNN) to classify images from the CIFAR-10 dataset. Unlike the standard Multi-Layer Perceptron (MLP) used in previous assignments, this architecture utilizes a convolutional layer with learnable filters to extract spatial features, followed by a hidden layer with ReLU activation.

2 Method

2.1 Mathematical Background

2.1.1 Patchify Layer

The network is a three-layer architecture designed for image classification on the CIFAR-10 dataset. The first layer is a convolutional "patchify" layer applied with a stride equal to the width of the filter f. This operation splits the input image X into a grid of non-overlapping sub-regions.

To achieve efficient computation, the convolution is implemented as a matrix multiplication. The input X is represented by a matrix M_X, where each row is a flattened sub-patch of the image. Given a bank of n_f filters, $F_{all} = [vec(F_1), vec(F_2), \ldots, vec(F_{n_f})]$, the forward pass is defined as:

$$H = M_X F_{all} \tag{1}$$

$$\mathbf{h} = \begin{pmatrix} vec(H_1) \\ \vdots \\ vec(H_{n_f}) \end{pmatrix} \tag{2}$$

$$\mathbf{x}_1 = \max(0, W_1\mathbf{h} + \mathbf{b}_1) \tag{3}$$

$$\mathbf{s} = W_2\mathbf{x}_1 + \mathbf{b}_2 \tag{4}$$

$$\mathbf{p} = \text{SoftMax}(s) = \frac{\exp(s)}{\mathbf{1}^T \exp(s)} \tag{5}$$

2.1.2 Cost Function and Regularization

The network was trained by minimizing the cost function J, which consists of the cross-entropy loss and an L_2 regularization term:

$$J(\mathbf{x}, y, \mathbf{W}, \mathbf{b}) = -\log(\mathbf{p}_y) + \lambda \left(\|F_{all}\|_F^2 + \sum_i \|W_i\|_F^2 \right) \tag{6}$$

2.2 Label Smoothing

To prevent overfitting in larger models, label smoothing is applied. This involves spreading probability from the ground-truth class to other classes, making the model less overconfident. The target vector y_{smooth} for a ground truth label y is defined as:

$$y_{smooth,i} = \begin{cases} 1 - \epsilon & \text{if } i = y \\ \epsilon/(K-1) & \text{otherwise} \end{cases} \tag{7}$$

2.3 Cyclical Learning Rates with Increasing Step Sizes

One of the models utilizes cyclical learning rates (CLR) where the learning rate η varies between η_{min} and η_{max}. To improve training efficiency for longer runs, the number of update steps per cycle (n_s) is doubled after each completed cycle:

$$n_{i+1,s} = 2n_{i,s} \tag{8}$$

This approach allows the model to escape local minima in early cycles and converge more precisely in later, longer cycles.

2.4 Gradient Verification

The analytical gradients computed in the `BackwardPass` were verified against the ground-truth values provided in the assignment's `debug_info.npz` file.

The relative error between the two results was calculated using:

$$\text{RelError} = \frac{|g_a - g_n|}{\max(\text{eps}, |g_a| + |g_n|)} \tag{9}$$

- **Filter Gradients (∇F_{all}):** 8.21×10^{-15}
- **Weight Gradients (∇W_1):** 2.50×10^{-14}
- **Weight Gradients (∇W_2):** 1.15×10^{-15}

All values are significantly low, meaning the gradients computed in `BackwardPass` are correct.

2.5 Data

The training data was shuffled at the start of each cycle to ensure that mini-batches were stochastic. The network parameters were initialized as follows:

- **Weights (W and F):** Initialized using a Gaussian distribution with $\mu = 0$ and a variance following the He initialization scheme, $\sigma = \sqrt{2/n_{in}}$, where n_{in} is the number of input units to the layer. This is specifically chosen to maintain signal variance through the ReLU activation functions.
- **Biases (b):** All bias vectors were initialized to zero.

3 Training Results

3.1 Train small networks with cyclic learning rates

Architecture 2 ($f = 4, n_f = 10, n_h = 50$) was evaluated over two cycles with a fixed n_s. As shown in Figure 1, the model achieved a final test accuracy of 55.94%. The training process exhibited the expected cyclical behavior in both loss and accuracy.



Figure 1: Training and validation metrics for Architecture 2 using a standard cyclical learning rate.

To find the optimal balance between spatial feature extraction and computational cost, four architectures were tested with varying filter sizes (f) and counts (n_f). The results are summarized in Figure 2.

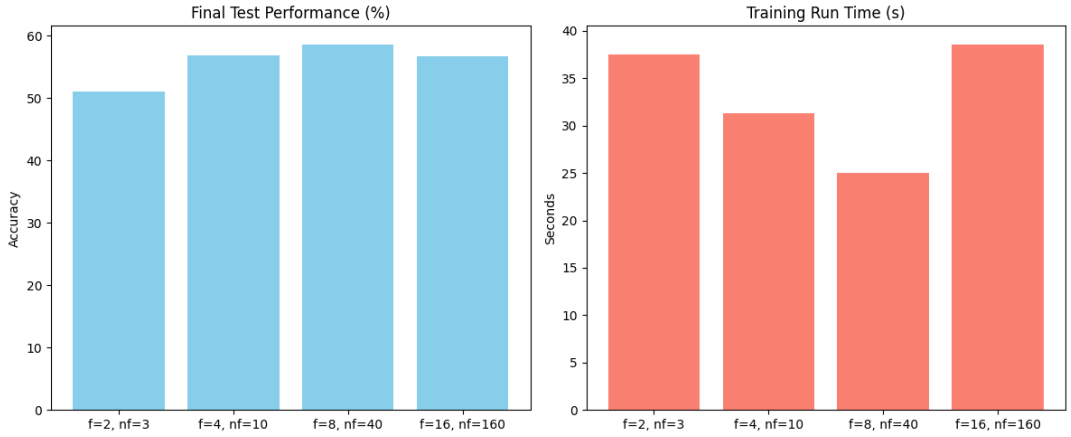


Figure 2: Performance and runtime comparison of four network architectures with varying filter sizes (f) and filter counts (n_f).

The two best performing networks was Architecture 2 and 3 which both showed the best test accuracy but also the fastest training time.

3.2 Increasing Cycle Lengths

To allow the best-performing models more time to converge at lower learning rates, an increasing cycle length schedule was implemented, doubling n_s after each cycle following Equation 8. This modification provided Architecture 2 and 3 more time.

Architecture 2 ($f = 4, n_f = 10$) reached an accuracy of 57.50% (Figure 3) and Architecture 3 ($f = 8, n_f = 40$) reached an accuracy of 60.93% (Figure 4).

To determine if the better performance in Net 3 was due to the larger filter size or the increased layer width, Architecture 2 was increased to $n_f = 40$. This extended version of Net 2 achieved a accuracy of 64.12% and can be seen in Figure 5. This suggested that total parameter count and width are more critical than the specific patch size f.

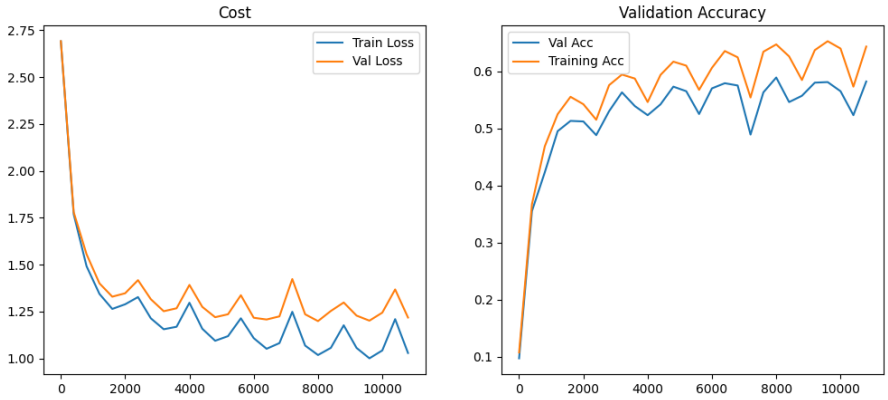


Figure 3: Architecture 2 ($f = 4$ and $n_f = 10$), Test accuracy: 57.50%

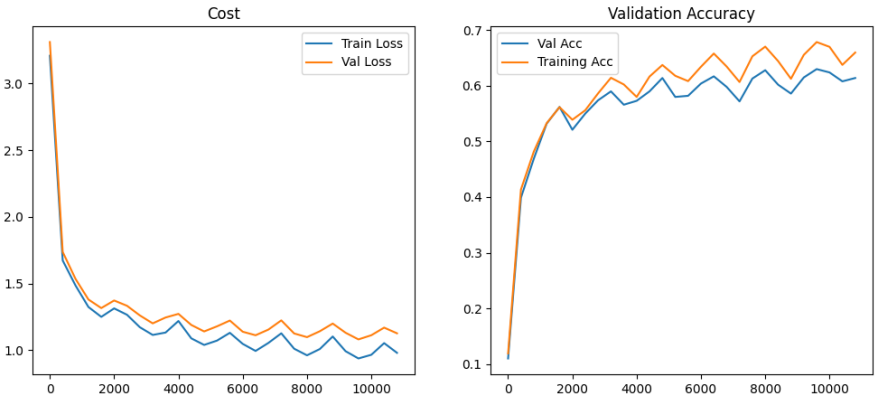


Figure 4: Architecture 2 ($f = 8$ and $n_f = 40$), Test accuracy: 60.93%



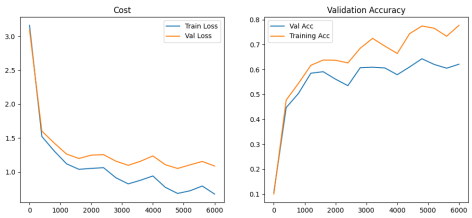
Figure 5: Architecture 2 with extra width ($f = 4$ and $n_f = 40$), Test accuracy: 64.12%

3.3 Larger networks and regularization with label smoothing

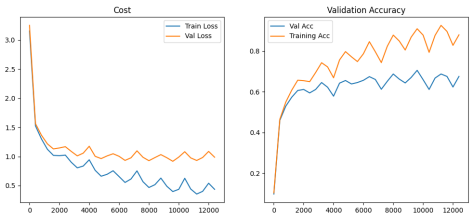
A larger network ($f = 4, n_f = 40, n_h = 300$) with $\lambda = 0.0025$ was trained to observe overfitting and the effects of regularization.

Without Label Smoothing:

The model reached a test accuracy of 65.12% but exhibited significant overfitting. As shown in Figure 6a, the validation loss began to diverge from the training loss, resulting in an average generalization gap of 0.3817. To make the overfitting even more obvious the same network was trained for 8 cycles instead of 4 and is displayed in Figure 6b. The overfitting became even more pronounced as training loss continued to drop while validation loss remained stagnant.



(a) Architecture 5 (4 cycles)



(b) Architecture 5 (8 cycles)

Figure 6: Visualizing overfitting in large networks without regularization.

With Label Smoothing ($\epsilon = 0.1$):

Implementing label smoothing following Equation 7 resulted in a final test accuracy of 65.04%. While the raw accuracy was comparable, the generalization gap was reduced to 0.2993. As visualized in Figure 7, the validation loss tracked the training loss more closely, indicating that label smoothing effectively prevents the model from becoming overconfident in its predictions and improves the stability of the training process.

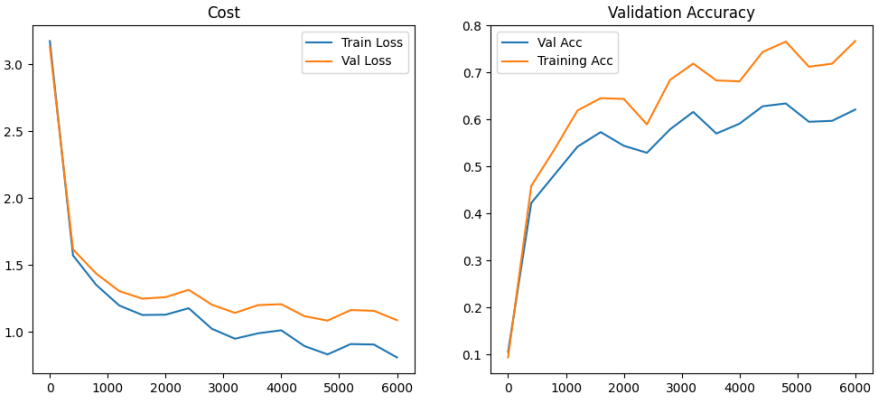


Figure 7: Training metrics for Architecture 5 with label smoothing

3.4 Boost performance

To maximize the classification accuracy, several enhancement techniques could be implemented:

- **Data Augmentation:** In previous assignments a horizontal flipping algorithm was applied to the training images. This would force the convolutional filters to learn features that are invariant to horizontal orientation. This increases the diversity of the training set and improves the model's ability to generalize.
- **Hyperparameter Search:** Given sufficient computational resources, a coarse-to-fine search could be performed, similar to the previous assignment, to systematically identify the optimal hyperparameters for the model.
- **Model Selection:** To determine the most effective network architecture like, optimal depth or layer size, k-fold cross-validation could be utilized. This provides a more robust estimate of performance and helps in selecting the parameters that best prevent overfitting.

4 Conclusion

Transitioning to a CNN architecture with a patchify layer provided a significant performance increase over standard MLPs, reaching a peak accuracy of over 65% on CIFAR-10. The experiments demonstrated that while larger models are more expressive, they are prone to severe overfitting. Label smoothing proved vital in reducing the generalization gap, while the implementation of increasing cycle lengths allowed wider architectures sufficient time to reach stable convergence.